SpatialForge 3D

Local-first 3D asset creation on AMD Radeon and ROCm

AMD AI DevMaster Hackathon 2026
Track 1: Development of Multimodal Content Creation Tools
Project Profile Document

Hardware	AMD Radeon Graphics, gfx1100 (Navi 31), 48 GB VRAM
Stack	ROCm 7.2.1, PyTorch 2.9.1, Python 3.12
Interface	Gradio web UI
Peak VRAM	12.97 GB of 48 GB (27%), all models resident
Cloud APIs used	None. Every inference stage runs locally.

1. Project background

Creating a usable 3D asset is still one of the slowest steps in game development, product visualisation and e-commerce. A single prop typically means hours of modelling, UV work and texturing, or a paid scanning service. Recent single-image reconstruction models have made the underlying task tractable in seconds, but almost every accessible implementation targets NVIDIA hardware and most consumer-facing tools route the work through a cloud API.

SpatialForge 3D was built to close both gaps at once. It turns a single photograph, or a sentence of text, into an interactive 3D mesh and a rendered turntable video. Every inference stage runs locally on an AMD Radeon GPU through ROCm. No image, no prompt and no geometry leaves the machine.

Doing this on ROCm was not a matter of changing a device string. Two core dependencies in the upstream pipelines are CUDA-only. Section 6 documents how each was resolved without giving up GPU execution of the models themselves.

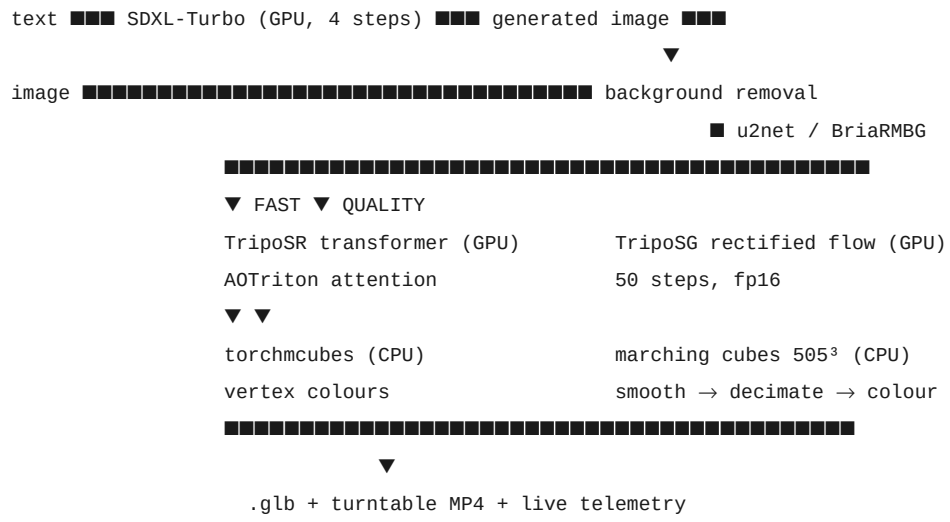
2. Target users and application scenarios

User	Scenario	Why local matters
Indie game developers	Prototype props and set dressing from reference photos or prompts.	No per-asset cost; iterate freely.
E-commerce sellers	Turn existing product photography into rotatable 3D listings and turntable clips.	Unreleased product imagery never leaves the building.
Product designers	Convert concept sketches and renders into inspectable 3D form.	Design IP stays on-premises.
Educators and hobbyists	Digitise physical objects with a phone camera.	Runs offline on a consumer Radeon card.

Track 1 creation tasks covered. Text-to-image (SDXL-Turbo generates the conditioning image from a prompt), image-to-video (an uploaded photograph becomes a rendered turntable MP4), and text-to-video (the two chained end to end).

3. System architecture

A single Gradio process holds every model resident in VRAM, so the load cost is paid once at startup rather than on each request. Two generation paths share the same front end, pre-processing and output stages.


text ■■■ SDXL-Turbo (GPU, 4 steps) ■■■ generated image ■■■
image ■■■

5. Measured performance on AMD Radeon

Every figure below was measured at runtime with `torch.cuda.synchronize()` around each stage and `torch.cuda.max_memory_allocated()` for VRAM. None are estimated, and the same telemetry is displayed live in the web UI on every generation.

Fast mode, 14-asset batch

Stage	Mean	Device
Background removal (u2net)	8 ms	CPU
TripoSR inference	1051 ms	GPU, AOTriton attention
Marching cubes	1332 ms	CPU fallback
GLB export	40 ms	CPU
End to end	2816 ms	

Inference ranged from 998 to 1389 ms across the 14 assets. Peak VRAM was **2.39 GB on every single asset**, with zero failures. That consistency, across subjects as different as a chair, a hamburger, a flamingo and an isometric building, is the clearest evidence of stable and diverse output.

Quality mode, single asset

Stage	Time	Device
Background removal (BriaRMBG)	225 ms	GPU
TripoSG rectified flow, 50 steps	42,965 ms	GPU
Marching cubes (ROCm CPU patch)	928 ms	CPU
Taubin smoothing, 4 iterations	699 ms	CPU
Quadric decimation, 3.4M to 400K faces	87,840 ms	CPU
Photo projection, silhouette fit 0.62	901 ms	CPU
GLB export	40 ms	CPU
Turntable render, 90 frames at 640p	54,307 ms	CPU
End to end	191,110 ms	

Peak VRAM reached **12.97 GB of 48 GB, or 27%**, with SDXL-Turbo, TripoSR, TripoSG and BriaRMBG all resident at once. The headroom matters commercially: the entire pipeline fits on consumer Radeon cards, not only workstation hardware.

Decimation dominates and scales directly with target face count. At the default 120K faces it falls to roughly 12 s, bringing end to end to about 90 s. The tradeoff is exposed as a slider rather than hidden.

6. AMD Radeon and ROCm adaptation

This is the part of the project that required real engineering rather than configuration.

6.1 CUDA-only dependencies

diso (TripoSG). TripoSG extracts its surface with `diso.DiffDMC`, a differentiable dual marching cubes implementation whose sources `#include <cuda_runtime.h>`. It cannot build against HIP; the wheel build fails outright. Rather than abandon TripoSG, the extraction step was replaced with scikit-image marching cubes

on CPU, preserving the original coordinate rescaling so output meshes are positioned identically. The transformer and VAE, which determine output quality, continue to run on the Radeon GPU. Measured cost: 928 ms at a 505^3 grid.

torchmcubes (TripoSR). Builds against HIP but its CUDA kernels are absent, so it reports `not compiled with CUDA support` and falls back to CPU automatically. Measured cost: 1332 ms.

cupy-cuda12x. Present in the base image and pulled in transitively by pymatting, breaking rembg on an AMD machine. Removing it lets pymatting use its numpy path.

6.2 What this means for the platform

Marching cubes is now the largest non-GPU stage in the Fast pipeline and a meaningful one in Quality. A HIP port of either torchmcubes or diso would remove roughly 1.3 s per Fast generation and 0.9 s per Quality generation, and would benefit every ROCm user of the wider 3D-generation ecosystem, not only this project. It is the clearest upstream contribution opportunity this work surfaced.

6.3 Platform findings

Three environment characteristics are documented in full in the repository README, since anyone reproducing this will hit them: the instance sits behind a TLS-intercepting proxy presenting a self-signed certificate, huggingface.co is unroutable and requires a mirror endpoint, and `/opt/venv` is not on persistent storage so an overlay virtual environment is needed for any package to survive an instance restart.

One positive finding worth recording: PyTorch on ROCm selected the AOTriton backend for efficient attention automatically, with no code changes. TripoSR's transformer benefited immediately.

7. Deliverables and output forms

Output	Format	Notes
Interactive 3D asset	.glb	Vertex-coloured, engine-ready
Turntable animation	.mp4, 640p 30fps	90 frames, software rendered
Live telemetry	In-UI panel	Per-stage timings, VRAM, mesh stats
Batch benchmark	.md and .csv	Reproducible via sf_bench.py
Web interface	Gradio	Publicly exposable via rc-tunnel

8. Honest limitations

Single-view reconstruction infers unseen surfaces rather than measuring them, so the back of an object is plausible but not accurate. Photo projection covers only the visible surface, typically 20 to 30 percent of vertices, with the remainder using model-predicted colour. Colour is stored per vertex rather than in a baked UV texture, so the finest detail is bounded by vertex density. Layered or thin-edged subjects such as food and foliage reconstruct less reliably than solid objects with clear silhouettes. Each of these is a known property of the approach, stated here rather than hidden.

9. Reproducibility

The repository contains a complete environment bootstrap script, the ROCm patch as a standalone runnable file, and a benchmark harness that regenerates every number in section 5. Following the README should allow an evaluator to reproduce the submitted results on the same Radeon Cloud image.

File	Purpose
`spatialforge_app.py`	Gradio UI, both modes, telemetry
`turntable.py`	Software turntable renderer, no OpenGL
`projtex.py`	Silhouette-matched photo projection
`patch_triposg.py`	ROCm patch replacing diso
`sf_bench.py`	Batch benchmark harness
`setup_session.sh`	Environment bootstrap

SpatialForge 3D. Built for the AMD AI DevMaster Hackathon 2026, Track 1. Source code MIT licensed; model weights remain under their respective licences.